

# Linear Regression Ultimate Cheat Sheet

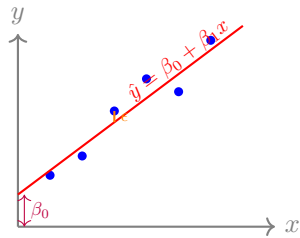
Building, Evaluating, and Diagnosing Models with Scikit-Learn

## 1 The Linear Model

### Equation

$$y = \beta_0 + \beta_1 x_1 + \dots + \beta_n x_n + \epsilon$$

- $y$ : Target (Dependent Variable).
- $x$ : Features (Independent Variables).
- $\beta_0$ : Intercept (Bias).
- $\beta_1$ : Slope (Weight/Coefficient).
- $\epsilon$ : Residual Error ( $y - \hat{y}$ ).



### Key Assumptions

1. **Linearity**: Relationship between  $X$  and  $y$  is linear.
2. **Independence**: Observations are independent.
3. **Homoscedasticity**: Constant variance of errors.
4. **Normality**: Errors are normally distributed.

## 2 Scikit-Learn Workflow

### 1. Setup & Load

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error,
mean_squared_error, r2_score
```

```
df = pd.read_csv("data.csv")
```

### 2. Split Data

```
X = df[['feature1', 'feature2']] # Features (2D)
y = df['target']                # Target (1D)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

### 3. Train Model

```
model = LinearRegression()
model.fit(X_train, y_train)

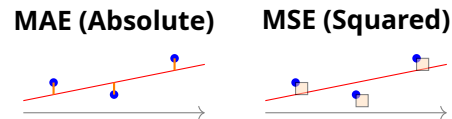
# Inspect Parameters
print(f"Intercept: {model.intercept_}")
print(f"Coefficients: {model.coef_}")
```

### 4. Predict

```
y_pred = model.predict(X_test)
```

## 3 Evaluation Metrics

How wrong is the model?



### 1. Mean Absolute Error (MAE)

$$MAE = \frac{1}{n} \sum |y_i - \hat{y}_i|$$

Average magnitude of error. Robust to outliers.

```
mae = mean_absolute_error(y_test, y_pred)
```

### 2. Mean Squared Error (MSE)

$$MSE = \frac{1}{n} \sum (y_i - \hat{y}_i)^2$$

Punishes large errors heavily (squaring). Hard to interpret (units<sup>2</sup>).

```
mse = mean_squared_error(y_test, y_pred)
```

### 3. Root Mean Squared Error (RMSE)

$$RMSE = \sqrt{MSE}$$

Same unit as target. Most common metric.

```
rmse = np.sqrt(mse)
```

### 4. R-Squared ( $R^2$ )

$$R^2 = 1 - \frac{RSS}{TSS}$$

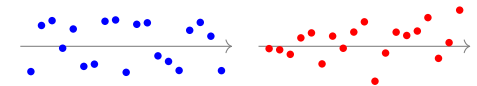
Percentage of variance explained by the model (0 to 1). Higher is better.

```
r2 = r2_score(y_test, y_pred)
```

## 4 Residual Analysis

Check if the model assumptions hold.

Good (Homoscedastic)    Bad (Heteroscedastic)



### Visual Checks

```
import matplotlib.pyplot as plt

residuals = y_test - y_pred

plt.scatter(y_pred, residuals)
plt.axhline(0, color='red', linestyle='--')
plt.xlabel("Predicted")
plt.ylabel("Residuals")
plt.title("Residual Plot")
plt.show()
```

### Interpretation

- **Random Cloud**: Good. Model captures the pattern.
- **Funnel Shape**: Heteroscedasticity (Variance changes). Scaling might help (Log transform  $y$ ).
- **Curve/Pattern**: Missing non-linear feature (Try Polynomials).

## 5 Improving the Model

### 1. Feature Engineering

- Log transform skewed features.
- Interaction terms ( $x_1 \cdot x_2$ ).

## 2. Regularization *Prevent overfitting by penalizing large coefficients.*

**Ridge (L2):** Shrinks coefs towards 0.

```
from sklearn.linear_model import Ridge
```

```
ridge = Ridge(alpha=1.0) # alpha = strength
ridge.fit(X_train, y_train)
```

**Lasso (L1):** Sets irrelevant coefs to 0 (Feature Selection).

```
from sklearn.linear_model import Lasso
lasso = Lasso(alpha=0.1)
lasso.fit(X_train, y_train)
```